Zevio

Frontend ↔ API connection — change report

Hyperlocal food delivery app for Papanasam ↔ Ayyampettai (hotels and bakeries)

Project folder	D:\projects\delivery website (frontend)
API used	D:\projects\zevio-apis-main (Flask + Supabase, live at zevio-apis.vercel.app)
Date	29/10/2026
Prepared for	Abdul Waheed.M
Scope chosen	“Connect what exists” — customer, shop panel and admin use the real API; parts the API does not support yet stay on demo data
API key	Kept on the server behind a proxy (never in the browser)
App name	Renamed Zovio → Zevio everywhere

Contents

- 1. Summary of the change
- 2. How the connection works (proxy and data modes)
- 3. How to run it — on your computer and on Netlify
- 4. Which screens now use the API, and which endpoints they call
- 5. How API data is mapped to the app (shops, products, orders, statuses)
- 6. Temporary workarounds for missing API columns
- 7. What still runs on demo data
- 8. Every file that was added or changed
- 9. Testing done (with screenshots)
- 10. Problems found in the API and what the backend must change
- 11. Known limits right now

1. Summary of the change

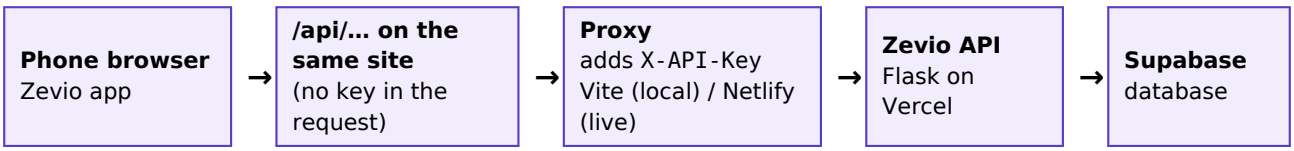
Before this change the app ran only on built-in demo data stored in the browser. Now it can talk to the Zevio API. One setting (**VITE_DATA_SOURCE**) decides which one the app uses, so the demo still works with no server.

- **Customer app** — login creates a real profile; shops, menus, search, placing an order, my orders, order tracking and cancel all use the API.
- **Shop panel** — shop login, new-order list (checks every 8 seconds), accept / ready / reject, products (add, edit, price, in-stock, delete), Open/Closed switch and shop settings use the API.
- **Admin panel** — dashboard numbers, all orders, and shops (add, edit, active on/off) use the API.
- **Delivery partner app** and a few admin lists stay on demo data, because the API has no endpoints for them yet (section 7).
- **The API key is safe** — it never goes to the browser. A small forwarder (proxy) adds it on the server side. This also removes the CORS problem.
- **No screen was rewritten.** Only the data layer (`src/services/`) changed, plus three small screen tweaks (section 8).
- **Name** — every “Zovio” is now “Zevio” (app title, loading screen, emails, UPI name, storage keys, package name).

Important: two bugs in the API code stop shop owners from adding shops or products (wrong-length ids). They are described in section 10 with the exact 2-line fix. The rest of the app works with the API as it is.

2. How the connection works

Path of every request



- The app always calls `/api/...` on its own address, for example `/api/shops/`.
- The proxy forwards it to the real API and adds the secret key. The key lives only in `.env.local` (your computer) or in Netlify’s environment variables (live site).
- Because the browser only talks to its own site, the API does not need CORS headers.
- The live proxy (Netlify) only lets through the 23 endpoints the app actually uses. Delete-user, delete-shop, delete-order and order-item endpoints are blocked.

Two data modes

Mode	How to turn on	Where the data comes from
Demo (default)	No .env.local file, or VITE_DATA_SOURCE=mock	Built-in sample shops, products and orders saved in the browser. No server needed. Good for showing the app.
API	VITE_DATA_SOURCE=api	The Zevio API. Parts the API does not support stay on demo data (section 7).

Each mode keeps its own login, cart and local data in the browser (keys start with `zevio.` in demo and `zevio.api.` in API mode), so a demo login can never be sent to the real API.

How the app decides per record

Records from the API have long UUID ids (e.g. `1ba99104-3056-4ac8-ba3b-d364528e9d70`). Demo records have short ids (e.g. `s1`, `o1012`). Every service function looks at the id: UUID → API, short id → demo data. That is how the delivery app can keep using demo orders while customers and shops use real ones.

3. How to run it

On your computer

1) In `D:\projects\delivery website`, copy `.env.example` to a new file named `.env.local`.

2) Open `.env.local` and fill it in:

```
VITE_DATA_SOURCE=api
ZEVIO_API_URL=https://zevio-apis.vercel.app
ZEVIO_API_KEY=<the API_KEY value from the API's secrets.env>
```

3) Run the app:

```
npm install
npm run dev
```

To test with the API running on your own computer, use `ZEVIO_API_URL=http://127.0.0.1:2222` and start the API with `python main.py`.

To go back to demo data, delete `.env.local` (or set `VITE_DATA_SOURCE=mock`) and restart `npm run dev`.

> **Never** commit or share `.env.local` — it holds the API key. It is already listed in `.gitignore`. The variables without the `VITE_` prefix are never put into the app's JavaScript; I checked the built files and the key is not in them.

On Netlify (live site)

- `netlify.toml` is already set: build command `npm run build`, publish folder `dist`, and `VITE_DATA_SOURCE=api`.
- In Netlify → Site configuration → Environment variables, add **ZEVIO_API_URL** = `https://zevio-apis.vercel.app` and **ZEVIO_API_KEY** = your key.
- Deploy. `netlify/edge-functions/api.js` handles every `/api/*` request and adds the key.
- `public/_redirects` still makes /shop-panel, /admin etc. work on refresh.

4. Screens that now use the API

Screen / action	API endpoint(s) called
Customer — login (name + phone)	POST /api/user/add/ (first login), GET /api/user/<id> (check), POST /api/user/update/ (name change)
Home — shop list, Hotels / Bakery filter, place filter	GET /api/shops/
Shop page — menu	GET /api/shop/<id>, GET /api/products/by-shop/<id>
Search (shops + items)	GET /api/shops/ + GET /api/products/by-shop/<id> for each shop (cached 30 s)
Checkout — place order (UPI or COD)	GET /api/shop/<id> + products (re-checks price, stock, open), POST /api/order/place/
My orders, Reorder	GET /api/orders/by-customer/<id>, GET /api/order/<id>/items/
Order tracking, Cancel	GET /api/order/<id>, POST /api/order/update-status/ (cancelled)
Shop panel — login (choose shop)	GET /api/shops/, GET /api/shop/<id>
New orders list (every 8 s, with sound)	GET /api/orders/by-shop/<id>, items per order (cached)
Accept / Ready / Reject	POST /api/order/update-status/
Open / Closed switch	POST /api/shop/toggle-orders/
Products — in-stock switch	POST /api/product/toggle-availability/
Products — add	POST /api/product/add/ then GET /api/product/slug/<slug> (to get the new id)
Products — edit / delete	POST /api/product/update/ (only changed fields), POST /api/product/delete/
Shop settings (phone, UPI, timings, address)	POST /api/shop/update/ (only changed fields)
Admin — dashboard, orders	GET /api/shops/ then GET /api/orders/by-shop/<id> for each shop
Admin — add shop	POST /api/user/add/ (owner profile), POST /api/shop/add/, GET /api/shop/slug/<slug>
Admin — edit shop, active on/off	POST /api/shop/update/

Errors from the API are shown as friendly messages: not found, network problem, “server could not do that”, and “could not connect to the Zevio server (API key)” when the key is wrong. Requests stop after 15 seconds and show the Retry screen.

5. How API data is mapped to the app

The API and the app use different field names. All conversion is in one file: `src/services/api/mappers.js`.

Shop

App field	API field
name	shop_name
phone / email	contact_phone / contact_email
address	address
placeId (papanasam, rajagiri…)	city (e.g. “Papanasam”) — matched to the service places by name
isOpen	is_accepting_orders
image	cover_image_url (or logo_url)
category, ownerName, upiId, timings, deliveryTime, isActive, subscription	no column yet → stored in description (section 6)

Product

App field	API field
name, price, image	name, price, image_url
inStock	is_available
section (menu group), isVeg	no column yet → stored in description

Order

App field	API field
number (shown as #ZV-…)	order_number
customerName / customerPhone	recipient_name / recipient_phone
itemTotal / deliveryFee / total	subtotal / delivery_fee / total_amount
items [name, price, qty]	order items: product_name, unit_price, quantity
address.street, landmark, directions; instructions; paymentMethod	delivery_address (packed text, section 6)
address.placeId	delivery_city
paymentStatus	worked out: UPI = paid; COD = paid when delivered
createdAt	created_at
partnerId, reject reason, later status times	no column yet → kept on the phone that made the change

Order status

App status	Saved in API as	Note
Placed	pending	
Accepted	confirmed	
Ready (for pickup)	preparing	API has no “ready”; matched step by step
Picked up	out_for_delivery	
Delivered	delivered	
Cancelled (by customer)	cancelled	
Rejected (by shop)	cancelled	API has no “rejected”; other phones see Cancelled

6. Temporary workarounds for missing API columns

Some information the app needs has no column in the database yet. So that every phone sees the same data, the app saves it inside existing text fields. It is readable in Supabase and is removed once the backend adds real columns (section 10).

Shop and product description

The description ends with one line that starts with `#zevio`, followed by the extra data. Example of a real shop saved during testing:

```
#zevio {"category":"hotels","ownerName":"Murugan","upiId":"murugan.new@okhdfc",
 "timings":{"open":"07:00","close":"22:00"},"deliveryTime":"25-35","isActive":true}
```

For products: `#zevio {"section":"Tiffin","isVeg":true}`. Any normal description text is kept above this line.

Shops created outside the app (with no #zevio line) still work: they show as Hotels, open 08:00–22:00, active, with no UPI ID until the owner saves the shop settings once.

Order delivery address

Parts are joined with “ | ”. Example of a real order saved during testing:

```
12, North Car Street | Landmark: Opp. temple | Note: Less spicy please | Pay: UPI
```

Tag	Meaning
(first part)	House no. / street
Landmark:	Landmark
Directions:	Extra directions from the saved address
Note:	Customer’s instructions for the shop
Pay:	UPI or COD

The shop owner sees all of this normally on the order screen (see the screenshot in section 9).

7. What still runs on demo data

Part	Why	What the API needs
Delivery partner app (orders, pickup, deliver, history)	No partner endpoints	assign-partner, orders by partner, online/offline
Assigning a partner (admin)	No column	delivery_partner_id on orders
Admin customers and partners lists, block customer	No list endpoint	GET /api/users/?user_type=…, is_blocked
Customer saved addresses	No addresses table	addresses table + endpoints (now saved on the phone)
Platform settings (delivery fee, places, platform UPI, support phone)	No settings endpoint	GET/POST /api/settings/
Photos	No upload	Image upload (e.g. Supabase Storage)

8. Files added and changed

New files

File	What it does
src/services/api/config.js	Reads VITE_DATA_SOURCE; tells if an id belongs to the API (UUID); makes UUIDs
src/services/api/client.js	Calls /api/… (GET, and POST as form fields), 15-second timeout, turns API errors into app error codes
src/services/api/mappers.js	All conversion between API and app: shops, products, orders, statuses, #zevio data, address packing, city ↔ place
src/services/api/apiShops.js	Shops and products: list, details, search, add, update only changed fields, open/closed, stock, delete; short caches
src/services/api/apiOrders.js	Orders: list by customer / shop / all, details with items, place order (re-checks prices), status changes, cancel
src/services/api/apiAuth.js	Customer login (creates/reuses an API profile) and shop login
netlify/edge-functions/api.js	Live proxy: adds the API key, allows only the endpoints the app uses
netlify.toml	Netlify build settings, turns on API mode
.env.example	Template for .env.local (data mode, API URL, key)
.gitignore	Keeps .env.local, node_modules and dist out of git
BACKEND-CHANGES.md	Full list of fixes, columns and endpoints for the API

Changed files

File	Change
src/services/shopService.js, productService.js, orderService.js, authService.js	Each function uses the API when the id is an API id (or API mode is on); otherwise the demo data as before
src/services/devService.js	DEV “simulate order” button creates a real test order in API mode
src/services/db.js	“Reset demo data” no longer deletes API logins, addresses and order notes
vite.config.js	Dev and preview proxy for /api that adds the key from .env.local
src/utils/constants.js	Separate storage keys for demo and API mode
src/components/admin/ShopFormDrawer.jsx	New Email field (required in API mode — the API needs it)
src/pages/customer/Login.jsx	The mock “Continue with Google” button is hidden in API mode (the API needs a phone number)
src/pages/customer/OrderTracking.jsx	DEV auto-progress is off by default in API mode, so testing never moves real orders by itself
src/pages/customer/Profile.jsx	Fixed a crash right after logging out
src/pages/admin/admin.module.css	Order numbers (#ZV-…) stay on one line
src/i18n/en.js	New messages (server error, API key problem, email); name Zevio
index.html, package.json, README.md, src/data/*.js, AdminLogin.jsx	Zovio → Zevio; README has the new “Connect to the Zevio API” section

9. Testing done

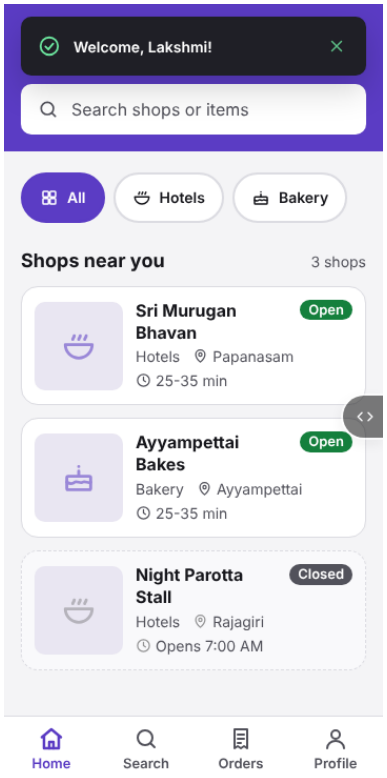
The live API (zevio-apis.vercel.app) could not be reached from my test machine, so I ran a copy of your API code locally (Flask, port 2222) with an in-memory stand-in for Supabase. The stand-in behaves like Postgres on ids (rejects non-UUID ids). I fixed the id bug in that copy only (not in your folder) so the full flow could run.

Test	Result
API mode, dev server — customer, shop owner and admin as three separate “phones”, 18 steps	**18 / 18 passed**, no console errors
API mode, production build through the proxy	All customer, shop and admin flows passed (4 steps not applicable: DEV-only button / leftover test data)
Demo mode — customer (19 steps), shop (13), delivery (9), admin (11)	**All passed**
Accessibility check (19 screens)	0 problems
Demo mode production smoke test (7 checks)	All passed, no console errors
API key search in the built JavaScript files	Key not found — it stays on the server

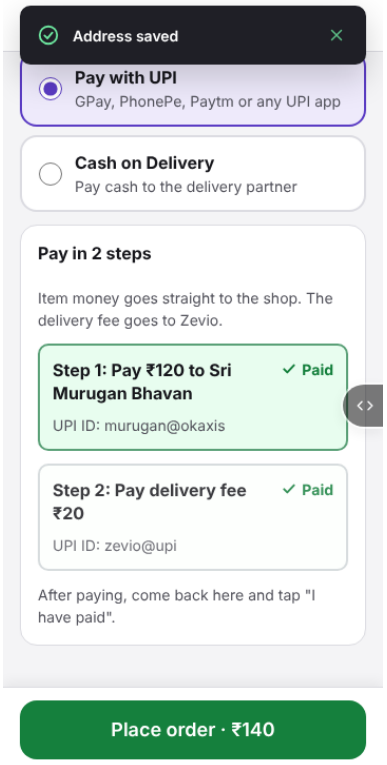
What the 18 API steps checked

- New customer logs in → a profile is created in the API.
- Hotels / Bakery filter works from the saved category.
- Add items, add address, UPI payment uses the **shop’s own UPI ID** from the API.
- Order placed → the **shop owner on another phone** sees it with instructions, landmark and payment method.
- Shop accepts → marks ready → the customer sees “Ready”.
- COD order → customer cancels.
- My orders list and search work on API data.
- Shop rejects an order with a reason.
- Shop: out-of-stock switch, add / edit price / delete a product.
- Shop changes UPI ID; closes the shop → the customer sees it closed; reopens.
- Admin dashboard and orders show API orders.
- Admin adds a shop (email required) and turns it off → customers no longer see it.

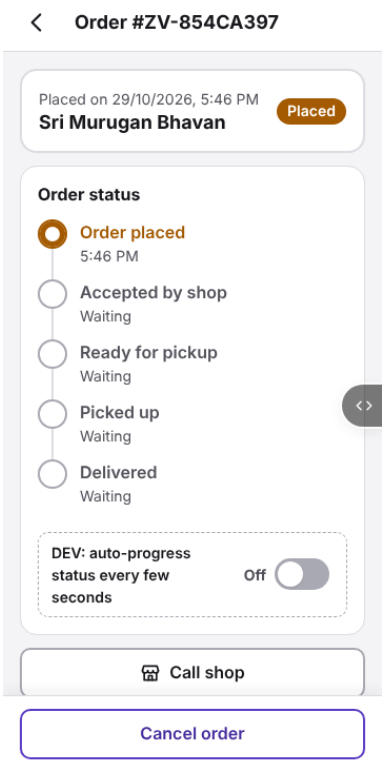
Screenshots from the API test (phone width 360 px)



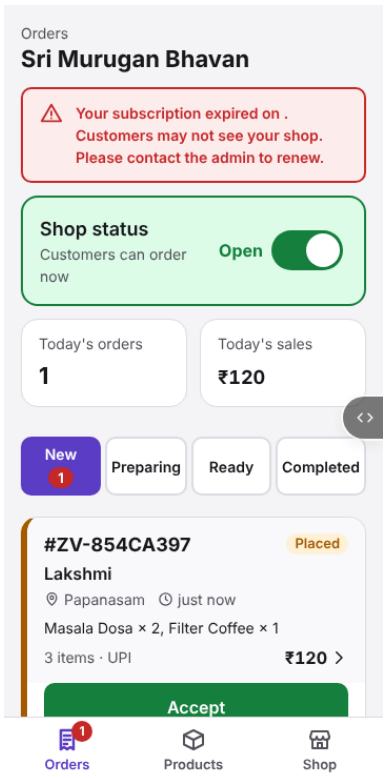
Home — shops from the API



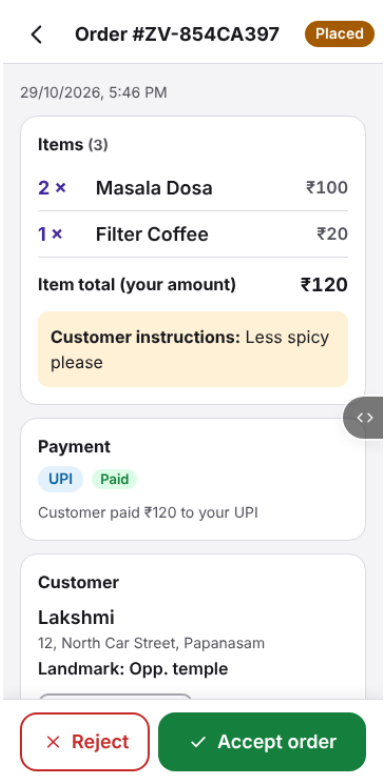
Checkout — pay the shop's UPI



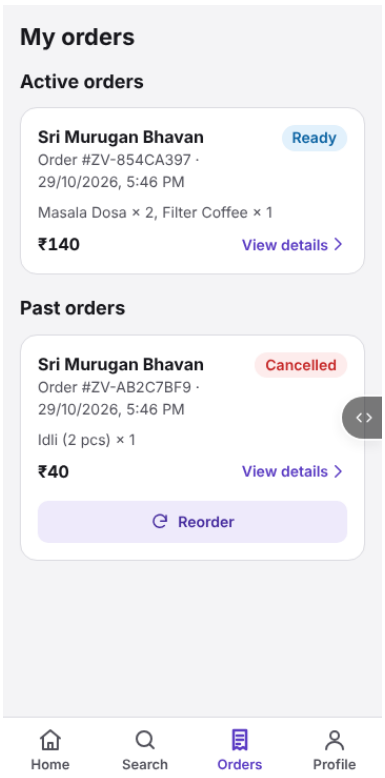
Tracking — order #ZV-...



Shop panel — new order arrived

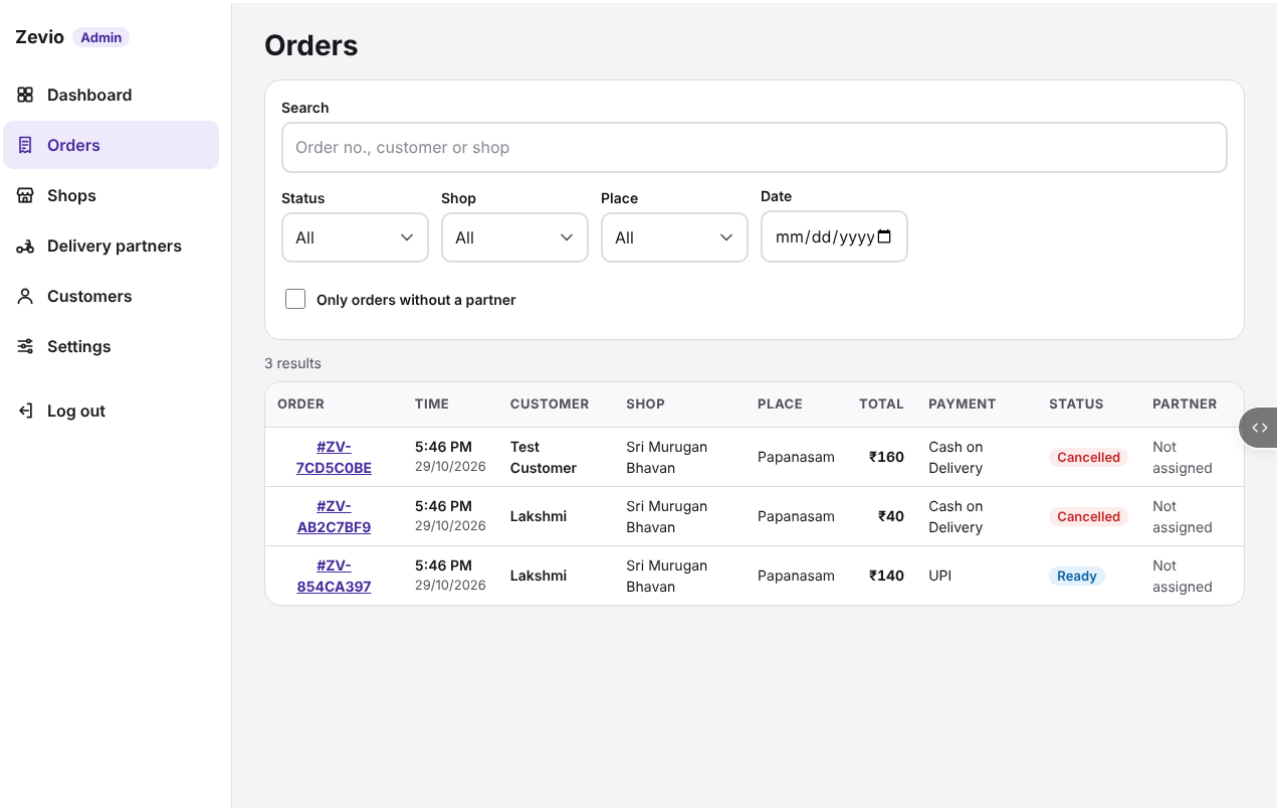


Shop — order with landmark, UPI



Customer — my orders

Admin — all orders from the API (desktop)



The order rejected by the shop shows as Cancelled for the admin, because the API has no “rejected” status yet (section 5).

10. Problems found in the API and what the backend must change

A. Bugs (fix first)

#	Problem	Effect	Fix
1	shop.py and product.py make ids with token_hex(8)-token_hex(4)-… → 16-8-8-8-24 characters, not a UUID	**Adding a shop or product fails**, or the new one cannot be edited (“invalid shop_id”)	from uuid import uuid4 → 'id': str(uuid4())
2	GET /api/user/<id> reads data[0] of an empty list	Server error (500) instead of “not found” (404)	return data[0] if data else None
3	add shop / add product do not return the new id	App needs an extra call (looks it up by slug)	Return {"id": new_id}
4	update endpoints store values without checks	Very long or empty values can be saved	Same checks as the add endpoints

B. Columns to add

- **shops:** category, upi_id, open_time, close_time, delivery_time, is_active, subscription plan / status / start / end
- **products:** section, is_veg
- **orders:** payment_method, payment_status, landmark, instructions, delivery_partner_id, reject_reason, accepted_at, ready_at, picked_up_at, delivered_at, cancelled_at
- **profiles:** is_blocked
- and allow them in the add / update / place-order endpoints. The full SQL is in BACKEND-CHANGES.md.

C. Statuses

Add **ready** and **rejected** to VALID_STATUSES, save the time of each change, and refuse impossible moves (e.g. delivered → pending).

D. Endpoints to add

- GET /api/user/by-phone/<phone> — so a customer can log in on a second phone
- Real sign-in (OTP or Google, e.g. Supabase Auth)
- GET /api/orders/ (all orders for admin) and order lists that include items
- GET /api/users/?user_type=… (admin lists)
- Delivery partner: assign-partner, orders by partner, online/offline
- Addresses, platform settings, image upload, mark COD paid

E. Security

- The key is now hidden, but anyone can still use the site’s public /api/… the way the app does. Real protection needs per-user sign-in in the API, so a shop owner can change only their own shop (Supabase Auth + Row Level Security).
- CORS is not needed while the proxy is used.
- The “csrf” value in every reply is random and never checked — remove it or implement it properly.

11. Known limits right now

- A customer who logs in on a new phone gets a new profile (the API cannot find people by phone yet). Their old orders stay with the old profile.
- Saved addresses, partner assignment, reject reasons and the times of later status changes are kept only on the phone that made them.
- “Ready” is saved as “preparing” and “Rejected” as “cancelled” in the database until the API adds those statuses.
- Admin “all orders” asks each shop for its orders one by one — fine for a few shops, slow with many.
- Photos are still preview-only (no upload in the API).
- Login is still a mock (no OTP). Shop owners choose their shop from a list.
- Everything was tested against a local copy of the API. Do one full order on the live API after fixing bug 1.

Full technical details for the backend: **BACKEND-CHANGES.md**. Setup steps: **README.md → “Connect to the Zevio API”**. Both are in the project folder.